

PAULT DOCUMENTATION

2026 02 19 Pro Expansion

Source: docs/plans/2026-02-19-pro-expansion.md

Theme: Clean Technical

Generated: February 26, 2026

Pro Tier Expansion Implementation Plan

> **For Claude:** REQUIRED SUB-SKILL: Use superpowers:executing-plans to implement this plan task-by-task.

Goal: Add four net-new Pro features to Pault: Usage Analytics (copy/run tracking), Prompt Versioning (snapshot history + diff + restore), Smart Collections (saved filters + AI-curated), and CLI Companion (pault binary for Terminal use).

Architecture: Features are built in dependency order. Analytics is first because it establishes the CopyEvent model and AnalyticsService that Smart Collections queries. Versioning adds PromptVersion snapshots triggered on save. Smart Collections adds SmartCollection sidebar entries that query PromptService and AIService. CLI Companion is a separate Xcode executable target sharing SwiftData store access.

Tech Stack: SwiftUI + SwiftData (macOS 15+), existing ProStatusManager/ProBadge/PaywallView for gating, existing AIService actor for AI-curated collections, existing PromptService service layer, ArgumentParser Swift package for CLI, XCTest for unit tests.

Reference: Existing Files

- Pault/Models/Prompt.swift — @Model with lastUsedAt: Date?, markAsUsed(), content, attributedContent
- Pault/Models/PromptRun.swift — @Model with prompt: Prompt?, createdAt, output, model, provider
- Pault/PaultApp.swift — sharedModelContainer with Schema([Prompt, Tag, TemplateVariable, Attachment, PromptRun])
- Pault/Services/PromptService.swift — copyToClipboard(_) calls prompt.markAsUsed() then saves
- Pault/Services/AIService.swift — actor with improve(prompt:config:), suggestVariables(prompt:config:), etc.
- Pault/Views/InspectorView.swift — segmented InspectorTab enum (.info, .history); 220pt wide
- Pault/Views/SidebarView.swift — SidebarFilter enum (.all, .recent, .archived, .tag(Tag))
- Pault/Services/ProStatusManager.swift — shared.isProUnlocked, shared.isTeamUnlocked
- Pault/Views/ProBadge.swift — reusable PRO badge component
- Pault/Views/PaywallView.swift — paywall sheet
- PaultTests/ — XCTest target; existing tests: PromptServiceTests, AIServiceTests, KeychainServiceTests

Phase 1: Usage Analytics

Tracks copy events locally. Adds CopyEvent model, AnalyticsService, Stats tab in Inspector, and Analytics sheet.

Task 1: CopyEvent Model

Files:

- Create: `Pault/Models/CopyEvent.swift`
- Modify: `Pault/PaultApp.swift` (register in Schema)
- Create: `PaultTests/Models/CopyEventTests.swift`

Step 1: Write the failing test

```
// PaultTests/Models/CopyEventTests.swift
import XCTest
import SwiftData
@testable import Pault

final class CopyEventTests: XCTestCase {
    var container: ModelContainer!

    override func setUpWithError() throws {
        let schema = Schema([CopyEvent.self])
        container = try ModelContainer(for: schema, configurations: [
            ModelConfiguration(isStoredInMemoryOnly: true)
        ])
    }

    func test_init_setsPromptIDAndTimestamp() throws {
        let ctx = ModelContext(container)
        let id = UUID()
        let before = Date()
        let event = CopyEvent(promptID: id)
        ctx.insert(event)
        try ctx.save()

        XCTAssertEqual(event.promptID, id)
        XCTAssertGreaterThanOrEqual(event.timestamp, before)
    }

    func test_init_generatesUniqueIDs() {
        let e1 = CopyEvent(promptID: UUID())
        let e2 = CopyEvent(promptID: UUID())
        XCTAssertNotEqual(e1.id, e2.id)
    }
}
```

Step 2: Run test to verify it fails

```
cd /Users/dev/Documents/Software/macOS/Pault
xcodebuild test -scheme Pault -destination 'platform=macOS' \
    -only-testing PaultTests/CopyEventTests 2>&1 | grep -E "error:|FAILED|PASSED|Build"
```

Expected: Build error — CopyEvent not found.

Step 3: Implement CopyEvent

```
// Pault/Models/CopyEvent.swift
import Foundation
import SwiftData

@Model
```

```
final class CopyEvent {
    var id: UUID
    var promptID: UUID
    var timestamp: Date

    init(promptID: UUID) {
        self.id = UUID()
        self.promptID = promptID
        self.timestamp = Date()
    }
}
```

Step 4: Register in ModelContainer

In `Pault/PaultApp.swift`, update the schema:

```
// Before:
let schema = Schema([
    Prompt.self,
    Tag.self,
    TemplateVariable.self,
    Attachment.self,
    PromptRun.self,
])

// After:
let schema = Schema([
    Prompt.self,
    Tag.self,
    TemplateVariable.self,
    Attachment.self,
    PromptRun.self,
    CopyEvent.self,
])
```

Step 5: Run test to verify it passes

```
xcodebuild test -scheme Pault -destination 'platform=macOS' \
-only-testing PaultTests/CopyEventTests 2>&1 | grep -E "error:|FAILED|PASSED"
```

Expected: PASSED

Step 6: Commit

```
git add Pault/Models/CopyEvent.swift Pault/PaultApp.swift
PaultTests/Models/CopyEventTests.swift
git commit -m "feat: add CopyEvent SwiftData model for analytics tracking"
```

Task 2: Record Copy Events in PromptService

Files:

- Modify: `Pault/Services/PromptService.swift`
- Modify: `PaultTests/PromptServiceTests.swift`

Step 1: Write the failing test

Add to `PaultTests/PromptServiceTests.swift`:

```
func test_copyToClipboard_insertsCopyEvent() throws {
    let prompt = Prompt(title: "Test", content: "Hello")
    modelContext.insert(prompt)

    let service = PromptService(modelContext: modelContext)
    service.copyToClipboard(prompt)

    let events = try modelContext.fetch(FetchDescriptor<CopyEvent>())
    XCTAssertEqual(events.count, 1)
    XCTAssertEqual(events.first?.promptID, prompt.id)
}

func test_copyToClipboard_multipleCopies_recordsEachEvent() throws {
    let prompt = Prompt(title: "Test", content: "Hello")
    modelContext.insert(prompt)

    let service = PromptService(modelContext: modelContext)
    service.copyToClipboard(prompt)
    service.copyToClipboard(prompt)
    service.copyToClipboard(prompt)

    let events = try modelContext.fetch(FetchDescriptor<CopyEvent>())
    XCTAssertEqual(events.count, 3)
}
```

Step 2: Run test to verify it fails

```
xcodebuild test -scheme Pault -destination 'platform=macOS' \
  -only-testing PaultTests/PromptServiceTests/test_copyToClipboard_insertsCopyEvent 2>&1 |
  grep -E "FAILED|PASSED"
```

Expected: FAILED

Step 3: Insert CopyEvent in copyToClipboard

In Pault/Services/PromptService.swift, add inside copyToClipboard(_ prompt:) right after prompt.markAsUsed():

```
// Insert copy tracking event
let copyEvent = CopyEvent(promptID: prompt.id)
modelContext.insert(copyEvent)
```

Step 4: Run tests to verify they pass

```
xcodebuild test -scheme Pault -destination 'platform=macOS' \
  -only-testing PaultTests/PromptServiceTests 2>&1 | grep -E "FAILED|PASSED"
```

Expected: All PASSED

Step 5: Commit

```
git add Pault/Services/PromptService.swift PaultTests/PromptServiceTests.swift
git commit -m "feat: record CopyEvent in PromptService.copyToClipboard"
```

Task 3: AnalyticsService

Files:

- Create: Pault/Services/AnalyticsService.swift
- Create: PaultTests/Services/AnalyticsServiceTests.swift

Step 1: Write the failing tests

```
// PaulTests/Services/AnalyticsServiceTests.swift
import XCTest
import SwiftData
@testable import Pault

final class AnalyticsServiceTests: XCTestCase {
    var container: ModelContainer!
    var ctx: ModelContext!

    override func setUpWithError() throws {
        let schema = Schema([Prompt.self, Tag.self, TemplateVariable.self,
            Attachment.self, PromptRun.self, CopyEvent.self])
        container = try ModelContainer(for: schema, configurations: [
            ModelConfiguration(isStoredInMemoryOnly: true)
        ])
        ctx = ModelContext(container)
    }

    func test_copyCount_returnsCorrectCount() throws {
        let id = UUID()
        for _ in 0..<5 {
            ctx.insert(CopyEvent(promptID: id))
        }
        try ctx.save()

        let service = AnalyticsService(modelContext: ctx)
        XCTAssertEqual(service.copyCount(for: id), 5)
    }

    func test_copyCount_excludesOtherPrompts() throws {
        let id1 = UUID()
        let id2 = UUID()
        ctx.insert(CopyEvent(promptID: id1))
        ctx.insert(CopyEvent(promptID: id1))
        ctx.insert(CopyEvent(promptID: id2))
        try ctx.save()

        let service = AnalyticsService(modelContext: ctx)
        XCTAssertEqual(service.copyCount(for: id1), 2)
        XCTAssertEqual(service.copyCount(for: id2), 1)
    }

    func test_topPromptsByUsage_sortsByTotalUsage() throws {
        let id1 = UUID()
        let id2 = UUID()
        // id1: 3 copies, id2: 5 copies
        for _ in 0..<3 { ctx.insert(CopyEvent(promptID: id1)) }
        for _ in 0..<5 { ctx.insert(CopyEvent(promptID: id2)) }
        try ctx.save()

        let service = AnalyticsService(modelContext: ctx)
        let top = service.topPromptIDsByUsage(limit: 10)
        XCTAssertEqual(top.first, id2)
        XCTAssertEqual(top.last, id1)
    }

    func test_lastUsed_returnsMaxTimestamp() throws {
```

```

let id = UUID()
let early = CopyEvent(promptID: id)
early.timestamp = Date(timeIntervalSinceNow: -3600)
let recent = CopyEvent(promptID: id)
recent.timestamp = Date()
ctx.insert(early)
ctx.insert(recent)
try ctx.save()

let service = AnalyticsService(modelContext: ctx)
let last = service.lastCopied(promptID: id)
XCTAssertNotNil(last)
XCTAssertEqual(last?.timeIntervalSince1970,
recent.timestamp.timeIntervalSince1970, accuracy: 1)
}
}

```

Step 2: Run test to verify it fails

```

xcodebuild test -scheme Pault -destination 'platform=macOS' \
-only-testing PaultTests/AnalyticsServiceTests 2>&1 | grep -E
"error:|FAILED|PASSED|Build"

```

Expected: Build error.

Step 3: Implement AnalyticsService

```

// Pault/Services/AnalyticsService.swift
import Foundation
import SwiftData
import os

private let analyticsLogger = Logger(subsystem: "com.pault.app", category: "analytics")

@MainActor
final class AnalyticsService {
private let modelContext: ModelContext

init(modelContext: ModelContext) {
self.modelContext = modelContext
}

// MARK: - Copy Stats

func copyCount(for promptID: UUID, since: Date? = nil) -> Int {
var descriptor = FetchDescriptor<CopyEvent>(
predicate: #Predicate { $0.promptID == promptID }
)
if let since {
descriptor = FetchDescriptor<CopyEvent>(
predicate: #Predicate { $0.promptID == promptID && $0.timestamp >= since }
)
}
return (try? modelContext.fetchCount(descriptor)) ?? 0
}

func lastCopied(promptID: UUID) -> Date? {
var descriptor = FetchDescriptor<CopyEvent>(
predicate: #Predicate { $0.promptID == promptID },

```

```

sortBy: [SortDescriptor(\.timestamp, order: .reverse)]
)
descriptor.fetchLimit = 1
return (try? modelContext.fetch(descriptor)?.first?.timestamp
}

// MARK: - Run Stats (from PromptRun)

func runCount(for prompt: Prompt) -> Int {
    let id = prompt.id
    let descriptor = FetchDescriptor<PromptRun>(
        predicate: #Predicate { $0.prompt?.id == id }
    )
    return (try? modelContext.fetchCount(descriptor)) ?? 0
}

// MARK: - Aggregate

/// Returns prompt IDs sorted by total usage (copies + runs), descending.
func topPromptIDsByUsage(limit: Int = 20) -> [UUID] {
    guard let events = try? modelContext.fetch(FetchDescriptor<CopyEvent>()) else { return
    [] }
    var counts: [UUID: Int] = [:]
    for event in events {
        counts[event.promptID, default: 0] += 1
    }
    return counts.sorted { $0.value > $1.value }
        .prefix(limit)
        .map(\.key)
}

/// Returns daily copy counts for a given prompt over the last N days.
func dailyCopies(for promptID: UUID, days: Int = 30) -> [(date: Date, count: Int)] {
    let since = Calendar.current.date(byAdding: .day, value: -days, to: Date()) ?? Date()
    let descriptor = FetchDescriptor<CopyEvent>(
        predicate: #Predicate { $0.promptID == promptID && $0.timestamp >= since },
        sortBy: [SortDescriptor(\.timestamp)]
    )
    guard let events = try? modelContext.fetch(descriptor) else { return [] }

    var result: [(date: Date, count: Int)] = []
    let calendar = Calendar.current
    var grouped: [Date: Int] = [:]
    for event in events {
        let day = calendar.startOfDay(for: event.timestamp)
        grouped[day, default: 0] += 1
    }
    for daysBack in (0..

```

Step 4: Run tests to verify they pass


```
xcodebuild test -scheme Pault -destination 'platform=macOS' \
-only-testing PaultTests/AnalyticsServiceTests 2>&1 | grep -E "FAILED|PASSED"
```

Expected: All PASSED

Step 5: Commit

```
git add Pault/Services/AnalyticsService.swift
PaultTests/Services/AnalyticsServiceTests.swift
git commit -m "feat: add AnalyticsService with copy/run stats queries"
```

Task 4: Stats Tab in InspectorView

Files:

- Modify: Pault/Views/InspectorView.swift
- Create: Pault/Views/PromptStatsView.swift

Step 1: Create PromptStatsView

```
// Pault/Views/PromptStatsView.swift
import SwiftUI
import SwiftData

struct PromptStatsView: View {
    @Environment(\.modelContext) private var modelContext
    let prompt: Prompt

    private var service: AnalyticsService {
        AnalyticsService(modelContext: modelContext)
    }

    var body: some View {
        ScrollView {
            VStack(alignment: .leading, spacing: 16) {
                statRow(
                    icon: "doc.on.doc",
                    label: "Copies",
                    value: "\(service.copyCount(for: prompt.id))"
                )
                statRow(
                    icon: "play.circle",
                    label: "Runs",
                    value: "\(service.runCount(for: prompt))"
                )
                if let last = service.lastCopied(promptID: prompt.id) {
                    statRow(
                        icon: "clock",
                        label: "Last Copied",
                        value: last.formatted(date: .abbreviated, time: .shortened)
                    )
                }
                statRow(
                    icon: "clock.badge.checkmark",
                    label: "Last Used",
                    value: prompt.lastUsedAt.map {
                        $0.formatted(date: .abbreviated, time: .shortened)
                    } ?? "Never"
                )
            }
        }
    }
}
```

```

Divider()

Text("Last 30 Days")
    .font(.caption)
    .foregroundColor(.secondary)

let daily = service.dailyCopies(for: prompt.id, days: 30)
let max = daily.map(\.count).max() ?? 1
HStack(alignment: .bottom, spacing: 2) {
    ForEach(daily.suffix(14), id: \.date) { entry in
        Rectangle()
            .fill(entry.count > 0 ? Color.accentColor : Color.secondary.opacity(0.2))
            .frame(width: 8, height: max > 0 ? CGFloat(entry.count) / CGFloat(max) * 40 + 2 : 2)
        }
    }
    .frame(height: 44)
}
.padding()
}
}

private func statRow(icon: String, label: String, value: String) -> some View {
    HStack {
        Label(label, systemImage: icon)
            .foregroundColor(.secondary)
        Spacer()
        Text(value)
            .font(.callout)
            .monospacedDigit()
    }
}
}
}

```

Step 2: Add Stats tab to InspectorView

In `Pault/Views/InspectorView.swift`, update the `InspectorTab` enum:

```

// Before:
enum InspectorTab: String, CaseIterable {
    case info = "Info"
    case history = "History"
}

// After:
enum InspectorTab: String, CaseIterable {
    case info = "Info"
    case stats = "Stats"
    case history = "History"
}

```

Add the `stats` case to the tab content switch:

```

// In the switch selectedTab { ... }
// Add after case .info:
case .stats:
    if ProStatusManager.shared.isProUnlocked {
        PromptStatsView(prompt: prompt)
    } else {

```

```
proGateView
}
```

Step 3: Build and manually verify

```
xcodebuild build -scheme Pault -destination 'platform=macOS' 2>&1 | grep -E
"error:|warning:|Build"
```

Expected: Build SUCCEEDED

Open the app → select a prompt → open Inspector → verify "Stats" tab appears between Info and History → copy the prompt → return to Stats tab → verify copy count increments.

Step 4: Commit

```
git add Pault/Views/PromptStatsView.swift Pault/Views/InspectorView.swift
git commit -m "feat: add Stats tab to InspectorView with copy/run analytics"
```

Task 5: Analytics Sheet (Top Prompts)

Files:

- Create: Pault/Views/AnalyticsView.swift
- Modify: Pault/Views/ContentView.swift (add toolbar button)

Step 1: Create AnalyticsView

```
// Pault/Views/AnalyticsView.swift
import SwiftUI
import SwiftData

struct AnalyticsView: View {
    @Environment(\.modelContext) private var modelContext
    @Environment(\.dismiss) private var dismiss
    @Query(sort: [SortDescriptor(\Prompt.title)]) private var allPrompts: [Prompt]

    @State private var topPromptIDs: [UUID] = []

    private var analyticsService: AnalyticsService {
        AnalyticsService(modelContext: modelContext)
    }

    private var rankedPrompts: [(prompt: Prompt, copies: Int, runs: Int)] {
        topPromptIDs.compactMap { id in
            guard let prompt = allPrompts.first(where: { $0.id == id }) else { return nil }
            return (
                prompt: prompt,
                copies: analyticsService.copyCount(for: id),
                runs: analyticsService.runCount(for: prompt)
            )
        }
    }

    var body: some View {
        VStack(spacing: 0) {
            // Header
            HStack {
                Text("Analytics")
            }
        }
    }
}
```

```

.font(.title2.bold())
Spacer()
Button("Done") { dismiss() }
.buttonStyle(.borderedProminent)
}
.padding()

Divider()

if rankedPrompts.isEmpty {
ContentUnavailableView(
"No Usage Data",
systemImage: "chart.bar",
description: Text("Copy or run prompts to see stats here.")
)
.frame(maxWidth: .infinity, maxHeight: .infinity)
} else {
Table(rankedPrompts, columns: {
TableColumn("Prompt") { row in
Text(row.prompt.title)
}
TableColumn("Copies") { row in
Text("\\(row.copies)")
.monospacedDigit()
}
TableColumn("Runs") { row in
Text("\\(row.runs)")
.monospacedDigit()
}
TableColumn("Total") { row in
Text("\\(row.copies + row.runs)")
.monospacedDigit()
.bold()
}
})
}
.frame(width: 560, height: 400)
.onAppear {
topPromptIDs = analyticsService.topPromptIDsByUsage(limit: 50)
}
}
}

```

Step 2: Add toolbar button to ContentView

In `Pault/Views/ContentView.swift`, add a toolbar button to open the Analytics sheet. Find the existing toolbar and add:

```

@State private var showingAnalytics = false

// In toolbar:
if ProStatusManager.shared.isProUnlocked {
Button {
showingAnalytics = true
} label: {
Image(systemName: "chart.bar")
}
}

```

```
.help("Usage Analytics")
.sheet(isPresented: $showingAnalytics) {
  AnalyticsView()
}
}
```

Step 3: Build and verify

```
xcodebuild build -scheme Pault -destination 'platform=macOS' 2>&1 | grep -E
"error:|Build"
```

Open app → chart.bar toolbar button → Analytics sheet opens → "Top Prompts" table shows usage.

Step 4: Commit

```
git add Pault/Views/AnalyticsView.swift Pault/Views/ContentView.swift
git commit -m "feat: add Analytics sheet with top prompts table"
```

Phase 2: Prompt Versioning

Snapshots prompt content on every save. Adds PromptVersion model, History tab, diff view, and restore.

Task 6: PromptVersion Model

Files:

- Create: Pault/Models/PromptVersion.swift
- Modify: Pault/PaultApp.swift
- Create: PaultTests/Models/PromptVersionTests.swift

Step 1: Write the failing test

```
// PaultTests/Models/PromptVersionTests.swift
import XCTest
import SwiftData
@testable import Pault

final class PromptVersionTests: XCTestCase {
  var container: ModelContainer!

  override func setUpWithError() throws {
    let schema = Schema([Prompt.self, Tag.self, TemplateVariable.self,
      Attachment.self, PromptRun.self, CopyEvent.self, PromptVersion.self])
    container = try ModelContainer(for: schema, configurations: [
      ModelConfiguration(isStoredInMemoryOnly: true)
    ])
  }

  func test_init_capturesPromptState() throws {
    let ctx = ModelContext(container)
    let prompt = Prompt(title: "Original", content: "First content")
    ctx.insert(prompt)

    let version = PromptVersion(prompt: prompt)
```

```

ctx.insert(version)
try ctx.save()

XCTAssertEqual(version.title, "Original")
XCTAssertEqual(version.content, "First content")
XCTAssertNotNil(version.savedAt)
XCTAssertNil(version.changeNote)
}

func test_init_withChangeNote() {
    let prompt = Prompt(title: "T", content: "C")
    let version = PromptVersion(prompt: prompt, changeNote: "Added CoT")
    XCTAssertEqual(version.changeNote, "Added CoT")
}
}

```

Step 2: Run test to verify it fails

```

xcodebuild test -scheme Pault -destination 'platform=macOS' \
-only-testing PaultTests/PromptVersionTests 2>&1 | grep -E "error:|FAILED|PASSED|Build"

```

Expected: Build error.

Step 3: Implement PromptVersion

```

// Pault/Models/PromptVersion.swift
import Foundation
import SwiftData

@Model
final class PromptVersion {
    var id: UUID
    @Relationship(deleteRule: .nullify) var prompt: Prompt?
    var promptID: UUID // snapshot – survives prompt deletion
    var title: String
    var content: String
    var attributedContent: Data?
    var savedAt: Date
    var changeNote: String?

    init(prompt: Prompt, changeNote: String? = nil) {
        self.id = UUID()
        self.prompt = prompt
        self.promptID = prompt.id
        self.title = prompt.title
        self.content = prompt.content
        self.attributedContent = prompt.attributedContent
        self.savedAt = Date()
        self.changeNote = changeNote
    }
}

```

Step 4: Register in ModelContainer

In Pault/PaultApp.swift, add PromptVersion.self to the schema:

```

let schema = Schema([
    Prompt.self,
    Tag.self,
    TemplateVariable.self,

```

```
Attachment.self,
PromptRun.self,
CopyEvent.self,
PromptVersion.self, // add this
])
```

Step 5: Run tests to verify they pass

```
xcodebuild test -scheme Pault -destination 'platform=macOS' \
-only-testing PaultTests/PromptVersionTests 2>&1 | grep -E "FAILED|PASSED"
```

Expected: PASSED

Step 6: Commit

```
git add Pault/Models/PromptVersion.swift Pault/PaultApp.swift
PaultTests/Models/PromptVersionTests.swift
git commit -m "feat: add PromptVersion SwiftData model for version history"
```

Task 7: Snapshot on Save in PromptService

Files:

- Modify: Pault/Services/PromptService.swift
- Modify: PaultTests/PromptServiceTests.swift

Step 1: Write the failing tests

Add to PaultTests/PromptServiceTests.swift:

```
func test_saveSnapshot_createsVersion() throws {
    let prompt = Prompt(title: "Title", content: "Content")
    modelContext.insert(prompt)
    try modelContext.save()

    let service = PromptService(modelContext: modelContext)
    service.saveSnapshot(for: prompt)

    let versions = try modelContext.fetch(FetchDescriptor<PromptVersion>())
    XCTAssertEqual(versions.count, 1)
    XCTAssertEqual(versions.first?.title, "Title")
    XCTAssertEqual(versions.first?.content, "Content")
}

func test_saveSnapshot_prunesOldVersionsBeyondLimit() throws {
    let prompt = Prompt(title: "T", content: "C")
    modelContext.insert(prompt)

    let service = PromptService(modelContext: modelContext)
    for i in 0..<55 {
        prompt.content = "Version \(i)"
        service.saveSnapshot(for: prompt, limit: 50)
    }

    let versions = try modelContext.fetch(FetchDescriptor<PromptVersion>())
    XCTAssertEqual(versions.count, 50)
}
```

Step 2: Run test to verify it fails

```
xcodebuild test -scheme Pault -destination 'platform=macOS' \
-only-testing PaultTests/PromptServiceTests/test_saveSnapshot_createsVersion 2>&1 | grep
-E "FAILED|PASSED|Build"
```

Expected: Build error — saveSnapshot not found.

Step 3: Implement saveSnapshot

Add to Pault/Services/PromptService.swift:

```
// MARK: - Versioning

func saveSnapshot(for prompt: Prompt, changeNote: String? = nil, limit: Int = 50) {
    let version = PromptVersion(prompt: prompt, changeNote: changeNote)
    modelContext.insert(version)

    // Prune oldest versions beyond limit
    var descriptor = FetchDescriptor<PromptVersion>(
        predicate: #Predicate { $0.promptID == prompt.id },
        sortBy: [SortDescriptor(\.savedAt, order: .reverse)]
    )
    if let versions = try? modelContext.fetch(descriptor), versions.count > limit {
        versions.dropFirst(limit).forEach { modelContext.delete($0) }
    }

    save("saveSnapshot")
}
```

Step 4: Run tests to verify they pass

```
xcodebuild test -scheme Pault -destination 'platform=macOS' \
-only-testing PaultTests/PromptServiceTests 2>&1 | grep -E "FAILED|PASSED"
```

Expected: All PASSED

Step 5: Commit

```
git add Pault/Services/PromptService.swift PaultTests/PromptServiceTests.swift
git commit -m "feat: add saveSnapshot to PromptService with pruning"
```

Task 8: Call saveSnapshot When Prompt Is Saved

Files:

- Modify: Pault/Views/EditPromptView.swift (or wherever updatedAt is set on save)

Step 1: Find save location

In Pault/Views/EditPromptView.swift, locate where `prompt.updatedAt = Date()` is set (likely in a `onChange` or explicit save button action). Add `saveSnapshot` before the save:

```
// Wherever the prompt save/update is triggered – look for prompt.updatedAt = Date()
// Add BEFORE the existing save logic:
let service = PromptService(modelContext: modelContext)
service.saveSnapshot(for: prompt)
// then the existing: prompt.updatedAt = Date()
```

Step 2: Build and verify

```
xcodebuild build -scheme Pault -destination 'platform=macOS' 2>&1 | grep -E
"error:|Build"
```


Open app → edit a prompt → save → check PromptVersion via debug print or next task's UI.

Step 3: Commit

```
git add Pault/Views/EditPromptView.swift
git commit -m "feat: snapshot prompt on save via saveSnapshot"
```

Task 9: Version History UI in InspectorView

Files:

- Create: Pault/Views/PromptVersionHistoryView.swift
- Create: Pault/Views/PromptDiffView.swift
- Modify: Pault/Views/InspectorView.swift

Step 1: Create PromptVersionHistoryView

```
// Pault/Views/PromptVersionHistoryView.swift
import SwiftUI
import SwiftData

struct PromptVersionHistoryView: View {
    @Environment(\.modelContext) private var modelContext
    @Bindable var prompt: Prompt
    @State private var selectedVersion: PromptVersion?
    @State private var showingDiff = false
    @State private var showingRestore = false

    private var versions: [PromptVersion] {
        let id = prompt.id
        let descriptor = FetchDescriptor<PromptVersion>(
            predicate: #Predicate { $0.promptID == id },
            sortBy: [SortDescriptor(\.savedAt, order: .reverse)]
        )
        return (try? modelContext.fetch(descriptor)) ?? []
    }

    var body: some View {
        if versions.isEmpty {
            ContentUnavailableView(
                "No History",
                systemImage: "clock.arrow.circlepath",
                description: Text("History is saved each time you edit this prompt.")
            )
        } else {
            List(versions, id: \.id, selection: $selectedVersion) { version in
                VStack(alignment: .leading, spacing: 2) {
                    Text(version.savedAt.formatted(date: .abbreviated, time: .shortened))
                        .font(.caption)
                        .foregroundColor(.secondary)
                    if let note = version.changeNote {
                        Text(note)
                            .font(.caption2)
                            .foregroundColor(.tertiary)
                    }
                }
                .tag(version)
            }
        }
    }
}
```

```

        .listStyle(.sidebar)
        .safeAreaInset(edge: .bottom) {
        HStack {
        Button("View Diff") {
        showingDiff = true
        }
        .disabled(selectedVersion == nil)
        Spacer()
        Button("Restore") {
        showingRestore = true
        }
        .disabled(selectedVersion == nil)
        .buttonStyle(.borderedProminent)
        }
        .padding(8)
        }
        .sheet(isPresented: $showingDiff) {
        if let version = selectedVersion {
        PromptDiffView(current: prompt.content, previous: version.content,
        savedAt: version.savedAt)
        }
        }
        .confirmationDialog(
        "Restore this version?",
        isPresented: $showingRestore,
        titleVisibility: .visible
        ) {
        Button("Restore", role: .destructive) {
        if let version = selectedVersion {
        restoreVersion(version)
        }
        }
        Button("Cancel", role: .cancel) {}
        } message: {
        Text("The current content will be replaced. A snapshot of the current state will be
        saved first.")
        }
        }
        }

private func restoreVersion(_ version: PromptVersion) {
    let service = PromptService(modelContext: modelContext)
    // Snapshot current state before restore
    service.saveSnapshot(for: prompt, changeNote: "Before restore to
    \(version.savedAt.formatted())")
    // Restore
    prompt.content = version.content
    prompt.attributedContent = version.attributedContent
    prompt.title = version.title
    prompt.updatedAt = Date()
    try? modelContext.save()
}
}

```

Step 2: Create PromptDiffView

```

// Pault/Views/PromptDiffView.swift
import SwiftUI

```

```

struct PromptDiffView: View {
@Environment(\.dismiss) private var dismiss
let current: String
let previous: String
let savedAt: Date

var body: some View {
VStack(spacing: 0) {
HStack {
Text("Version from \(savedAt.formatted(date: .abbreviated, time: .shortened))")
.font(.headline)
Spacer()
Button("Done") { dismiss() }
}
.padding()
Divider()
HSplitView {
VStack(alignment: .leading) {
Label("Previous", systemImage: "clock")
.font(.caption.bold())
.foregroundColor(.secondary)
.padding(.horizontal)
.padding(.top, 8)
ScrollView {
Text(previous)
.font(.body.monospaced())
.padding()
.frame(maxWidth: .infinity, alignment: .leading)
}
}
VStack(alignment: .leading) {
Label("Current", systemImage: "doc.text")
.font(.caption.bold())
.foregroundColor(.secondary)
.padding(.horizontal)
.padding(.top, 8)
ScrollView {
Text(current)
.font(.body.monospaced())
.padding()
.frame(maxWidth: .infinity, alignment: .leading)
}
}
}
.frame(width: 700, height: 450)
}
}

```

Step 3: Add Versions tab to InspectorView

In Pault/Views/InspectorView.swift, update InspectorTab:

```

enum InspectorTab: String, CaseIterable {
case info = "Info"
case stats = "Stats"
case versions = "History"
case history = "Runs"

```

```
}
```

Add versions case to the content switch:

```
case .versions:
if ProStateManager.shared.isProUnlocked {
PromptVersionHistoryView(prompt: prompt)
} else {
proGateView
}
```

Note: rename the existing history tab raw value from "History" to "Runs" to avoid confusion with the new version history.

Step 4: Build and verify

```
xcodebuild build -scheme Pault -destination 'platform=macOS' 2>&1 | grep -E
"error:|Build"
```

Edit a prompt several times → Inspector → "History" tab → verify version list appears → select a version → "View Diff" → side-by-side diff renders → "Restore" → content reverts.

Step 5: Commit

```
git add Pault/Views/PromptVersionHistoryView.swift \
Pault/Views/PromptDiffView.swift \
Pault/Views/InspectorView.swift
git commit -m "feat: add version history tab with diff view and restore to
InspectorView"
```

Phase 3: Smart Collections

Dynamic sidebar sections: saved filter bookmarks + AI-curated semantic clusters.

Task 10: SmartCollection Model

Files:

- Create: Pault/Models/SmartCollection.swift
- Modify: Pault/PaultApp.swift
- Create: PaultTests/Models/SmartCollectionTests.swift

Step 1: Write the failing test

```
// PaultTests/Models/SmartCollectionTests.swift
import XCTest
import SwiftData
@testable import Pault

final class SmartCollectionTests: XCTestCase {
var container: ModelContainer!

override func setUpWithError() throws {
let schema = Schema([SmartCollection.self])
container = try ModelContainer(for: schema, configurations: [
ModelConfiguration(isStoredInMemoryOnly: true)
```

```

    })
  }

  func test_savedFilter_storesFilterJSON() throws {
    let ctx = ModelContext(container)
    let filter = SmartCollectionFilter(tags: [], onlyFavorites: true, recentDays: 7)
    let collection = SmartCollection(name: "Faves", icon: "star.fill", filter: filter)
    ctx.insert(collection)
    try ctx.save()

    XCTAssertEqual(collection.name, "Faves")
    XCTAssertEqual(collection.ruleType, .savedFilter)
    XCTAssertFalse(collection.filterJSON.isEmpty)
  }

  func test_aiCurated_storesPromptIDs() throws {
    let ctx = ModelContext(container)
    let ids = [UUID(), UUID(), UUID()]
    let collection = SmartCollection(name: "Coding", icon:
    "chevron.left.forwardslash.chevron.right", promptIDs: ids)
    ctx.insert(collection)
    try ctx.save()

    XCTAssertEqual(collection.ruleType, .aiCurated)
    XCTAssertEqual(collection.promptIDs.count, 3)
  }
}

```

Step 2: Run test to verify it fails

```

xcodebuild test -scheme Pault -destination 'platform=macOS' \
-only-testing PaultTests/SmartCollectionTests 2>&1 | grep -E "error:|Build"

```

Expected: Build error.

Step 3: Implement SmartCollection

```

// Pault/Models/SmartCollection.swift
import Foundation
import SwiftData

enum CollectionRuleType: String, Codable {
    case savedFilter
    case aiCurated
}

struct SmartCollectionFilter: Codable {
    var tagIDs: [UUID]
    var onlyFavorites: Bool
    var recentDays: Int? // nil = no recency filter

    init(tags: [Tag] = [], onlyFavorites: Bool = false, recentDays: Int? = nil) {
        self.tagIDs = tags.map(\.id)
        self.onlyFavorites = onlyFavorites
        self.recentDays = recentDays
    }
}

@Model

```

```

final class SmartCollection {
    var id: UUID
    var name: String
    var icon: String
    var sortOrder: Int
    var ruleType: CollectionRuleType
    var filterJSON: String // JSON-encoded SmartCollectionFilter for .savedFilter
    var promptIDs: [UUID] // cached prompt IDs for .aiCurated
    var createdAt: Date
    var lastRefreshed: Date?

    // Saved filter initializer
    init(name: String, icon: String, filter: SmartCollectionFilter, sortOrder: Int = 0) {
        self.id = UUID()
        self.name = name
        self.icon = icon
        self.sortOrder = sortOrder
        self.ruleType = .savedFilter
        self.filterJSON = (try? String(data: JSONEncoder().encode(filter), encoding: .utf8)) ?? "{}"
        self.promptIDs = []
        self.createdAt = Date()
    }

    // AI-curated initializer
    init(name: String, icon: String, promptIDs: [UUID], sortOrder: Int = 0) {
        self.id = UUID()
        self.name = name
        self.icon = icon
        self.sortOrder = sortOrder
        self.ruleType = .aiCurated
        self.filterJSON = "{}"
        self.promptIDs = promptIDs
        self.createdAt = Date()
    }

    var filter: SmartCollectionFilter? {
        guard ruleType == .savedFilter,
            let data = filterJSON.data(using: .utf8) else { return nil }
        return try? JSONDecoder().decode(SmartCollectionFilter.self, from: data)
    }
}

```

Step 4: Register in ModelContainer

Add `SmartCollection.self` to the schema in `Pault/PaultApp.swift`.

Step 5: Run tests and commit

```

xcodebuild test -scheme Pault -destination 'platform=macOS' \
  -only-testing PaultTests/SmartCollectionTests 2>&1 | grep -E "FAILED|PASSED"
git add Pault/Models/SmartCollection.swift Pault/PaultApp.swift
PaultTests/Models/SmartCollectionTests.swift
git commit -m "feat: add SmartCollection SwiftData model"

```

Task 11: AI Clustering in AIService

Files:

- Modify: Pault/Services/AIService.swift
- Modify: PaultTests/AIServiceTests.swift

Step 1: Add clusterPrompts method to AIService

```
// In AIService.swift, add:

struct CollectionSuggestion: Codable {
    var name: String
    var icon: String // SF Symbol name
    var promptTitles: [String]
}

func clusterPrompts(titles: [String], config: AIConfig) async throws ->
[CollectionSuggestion] {
    let titleList = titles.enumerated().map { "\($0.offset + 1). \($0.element)"
    }.joined(separator: "\n")
    let system = ""
    You are a prompt organization expert. Group the following prompts into 3-6 thematic
    collections.
    Return ONLY a JSON array. Each object must have:
    - "name": short collection name (2-4 words)
    - "icon": an SF Symbol name that fits the theme (e.g. "pencil", "code",
    "magnifyingglass")
    - "promptTitles": array of prompt titles from the input list that belong to this
    collection
    Do not include prompts in multiple collections. Every prompt must appear exactly once.
    ""
    let response = try await complete(system: system, user: titleList, config: config)
    guard let data = response.data(using: .utf8),
    let suggestions = try? JSONDecoder().decode([CollectionSuggestion].self, from: data)
    else {
        throw AIError.parseError(response.data(using: .utf8) ?? Data())
    }
    return suggestions
}
```

Step 2: Commit

```
git add Pault/Services/AIService.swift
git commit -m "feat: add clusterPrompts to AIService for AI-curated Smart Collections"
```

Task 12: Smart Collections Sidebar Section**Files:**

- Create: Pault/Views/SmartCollectionEditorView.swift
- Modify: Pault/Views/SidebarView.swift
- Modify: Pault/Views/ContentView.swift (add .smartCollection(SmartCollection) to SidebarFilter)

Step 1: Add smartCollection to SidebarFilter

In Pault/Views/SidebarView.swift, update the enum:

```
enum SidebarFilter: Hashable {
    case all
```

```

case recent
case archived
case tag(Tag)
case smartCollection(SmartCollection)
}

```

Step 2: Create SmartCollectionEditorView

```

// Pault/Views/SmartCollectionEditorView.swift
import SwiftUI
import SwiftData

struct SmartCollectionEditorView: View {
    @Environment(\.modelContext) private var modelContext
    @Environment(\.dismiss) private var dismiss
    @Query(sort: [SortDescriptor(\Tag.name)]) private var allTags: [Tag]
    @Query(sort: [SortDescriptor(\Prompt.title)]) private var allPrompts: [Prompt]

    @State private var name: String = ""
    @State private var icon: String = "folder"
    @State private var selectedTagIDs: Set<UUID> = []
    @State private var onlyFavorites = false
    @State private var recentDays: Int? = nil
    @State private var isGenerating = false
    @State private var generationError: String?

    var body: some View {
        NavigationStack {
            Form {
                Section("Name & Icon") {
                    TextField("Collection name", text: $name)
                    TextField("SF Symbol name", text: $icon)
                }

                Section("Filter Rules") {
                    Toggle("Favorites only", isOn: $onlyFavorites)
                    Picker("Last used within", selection: $recentDays) {
                        Text("Any time").tag(nil as Int?)
                        Text("7 days").tag(7 as Int?)
                        Text("30 days").tag(30 as Int?)
                    }
                }
                if !allTags.isEmpty {
                    ForEach(allTags) { tag in
                        Toggle(tag.name, isOn: Binding(
                            get: { selectedTagIDs.contains(tag.id) },
                            set: { if $0 { selectedTagIDs.insert(tag.id) } else { selectedTagIDs.remove(tag.id) } }
                        ))
                    }
                }
            }

            Section {
                Button(action: createSavedFilter) {
                    Label("Save as Filter Collection", systemImage: "folder.badge.plus")
                }
                .disabled(name.isEmpty)

                Button(action: generateWithAI) {

```



```
if isGenerating {
    ProgressView()
    .progressViewStyle(.circular)
    .scaleEffect(0.7)
} else {
    Label("Generate with AI", systemImage: "sparkles")
}
}
.disabled(isGenerating || allPrompts.isEmpty)
}

if let error = generationError {
    Section {
        Text(error).foregroundColor(.red).font(.caption)
    }
}

.navigationTitle("New Collection")
.toolbar {
    ToolbarItem(placement: .cancellationAction) {
        Button("Cancel") { dismiss() }
    }
}
.frame(width: 340, height: 480)
}

private func createSavedFilter() {
    let tags = allTags.filter { selectedTagIDs.contains($0.id) }
    let filter = SmartCollectionFilter(tags: tags, onlyFavorites: onlyFavorites, recentDays: recentDays)
    let collection = SmartCollection(name: name, icon: icon, filter: filter)
    modelContext.insert(collection)
    try? modelContext.save()
    dismiss()
}

private func generateWithAI() {
    isGenerating = true
    generationError = nil
    Task {
        do {
            let config = AIService.currentConfig()
            let titles = allPrompts.prefix(100).map(\.title)
            let suggestions = try await AIService.shared.clusterPrompts(titles: Array(titles), config: config)
            await MainActor.run {
                for (i, suggestion) in suggestions.enumerated() {
                    let ids = allPrompts
                        .filter { suggestion.promptTitles.contains($0.title) }
                        .map(\.id)
                    let col = SmartCollection(name: suggestion.name, icon: suggestion.icon, promptIDs: ids, sortOrder: i)
                    col.lastRefreshed = Date()
                    modelContext.insert(col)
                }
                try? modelContext.save()
                isGenerating = false
            }
        }
    }
}
```

```

dismiss()
}
} catch {
await MainActor.run {
generationError = error.localizedDescription
isGenerating = false
}
}
}
}
}
}
}
}

```

Step 3: Add Collections section to SidebarView

In `Pault/Views/SidebarView.swift`, add after the Tags Divider:

```

// Collections section
@Query(sort: [SortDescriptor(\SmartCollection.sortOrder)]) private var collections:
[SmartCollection]
@State private var showingNewCollection = false

// In body, after the tag rows:
if ProStateManager.shared.isProUnlocked {
if !collections.isEmpty {
Divider().padding(.vertical, 4)
Text("Collections")
.font(.caption.bold())
.foregroundColor(.secondary)
.padding(.leading, 12)

ForEach(collections) { collection in
FilterRow(
title: collection.name,
icon: collection.icon,
count: nil,
isSelected: selectedFilter == .smartCollection(collection)
) {
selectedFilter = .smartCollection(collection)
}
}
}

Button(action: { showingNewCollection = true }) {
Label("New Collection", systemImage: "plus")
.font(.caption)
.foregroundColor(.secondary)
}
.buttonStyle(.plain)
.padding(.leading, 12)
.sheet(isPresented: $showingNewCollection) {
SmartCollectionEditorView()
}
}
}

```

Step 4: Handle `.smartCollection` filter in `PromptService.filterPrompts`

In `Pault/Services/PromptService.swift`, add a new method:

```
func filterPrompts(_ prompts: [Prompt], collection: SmartCollection) -> [Prompt] {
    switch collection.ruleType {
    case .aiCurated:
        let ids = Set(collection.promptIDs)
        return prompts.filter { ids.contains($0.id) }
    case .savedFilter:
        guard let filter = collection.filter else { return [] }
        var result = prompts.filter { !$0.isArchived }
        if filter.onlyFavorites { result = result.filter(\.isFavorite) }
        if let days = filter.recentDays {
            let cutoff = Calendar.current.date(byAdding: .day, value: -days, to: Date()) ?? Date()
            result = result.filter { ($0.lastUsedAt ?? .distantPast) >= cutoff }
        }
        if !filter.tagIDs.isEmpty {
            let ids = Set(filter.tagIDs)
            result = result.filter { $0.tags.contains(where: { ids.contains($0.id) }) }
        }
        return result
    }
}
```

Step 5: Build and verify

```
xcodebuild build -scheme Pault -destination 'platform=macOS' 2>&1 | grep -E
"error:|Build"
```

Open app → sidebar "New Collection" button → create a saved filter → appears in sidebar → click it → prompts filter correctly.

Step 6: Commit

```
git add Pault/Views/SmartCollectionEditorView.swift \
Pault/Views/SidebarView.swift \
Pault/Services/PromptService.swift
git commit -m "feat: add Smart Collections sidebar section with saved filters and
AI-curated mode"
```

Phase 4: CLI Companion

A standalone pault binary that reads the SwiftData store and calls the AI provider.

Task 13: PaultCLI Xcode Target

Files:

- Create: PaultCLI/ directory and Swift source files
- Modify: Xcode project to add executable target (manual Xcode step)

Step 1: Create CLI entry point

```
// PaultCLI/main.swift
import Foundation
import ArgumentParser

@main
struct PaultCLI: ParsableCommand {
    static let configuration = CommandConfiguration(
```

```
commandName: "pault",
abstract: "Pault prompt library – terminal access",
version: "1.0.0",
subcommands: [ListCommand.self, GetCommand.self, CopyCommand.self, RunCommand.self]
)
}
```

Step 2: Create ListCommand

```
// PaultCLI/Commands/ListCommand.swift
import Foundation
import ArgumentParser
import SwiftData

struct ListCommand: ParsableCommand {
    static let configuration = CommandConfiguration(
        commandName: "list",
        abstract: "List prompts in the library"
    )

    @Option(name: .shortAndLong, help: "Filter by tag name")
    var tag: String?

    @Flag(name: .shortAndLong, help: "Show only favorites")
    var favorites = false

    @Flag(name: .long, help: "Output as JSON")
    var json = false

    func run() throws {
        let store = try PaultStore()
        var prompts = store.prompts.filter { !$0.isArchived }
        if let tag { prompts = prompts.filter { $0.tags.contains(where: { $0.name == tag }) } }
        if favorites { prompts = prompts.filter(\.isFavorite) }

        if json {
            let output = prompts.map { ["id": $0.id.uuidString, "title": $0.title] }
            let data = try JSONSerialization.data(withJSONObject: output, options: .prettyPrinted)
            print(String(data: data, encoding: .utf8) ?? "")
        } else {
            for prompt in prompts {
                print(prompt.title)
            }
        }
    }
}
```

Step 3: Create GetCommand

```
// PaultCLI/Commands/GetCommand.swift
import Foundation
import ArgumentParser

struct GetCommand: ParsableCommand {
    static let configuration = CommandConfiguration(
        commandName: "get",
        abstract: "Print prompt content"
    )
}
```

```

@Argument(help: "Prompt title (partial match supported)")
var title: String

@Flag(help: "Resolve template variables")
var resolve = false

@Option(name: .customLong("var"), parsing: .upToNextOption, help: "Variable bindings:
key=value")
var variables: [String] = []

func run() throws {
    let store = try PaulStore()
    guard let prompt = store.findPrompt(title: title) else {
        throw ValidationError("No prompt found matching '\(title)')")
    }
    var content = prompt.content
    if resolve {
        var bindings: [String: String] = [:]
        for pair in variables {
            let parts = pair.split(separator: "=", maxSplits: 1)
            if parts.count == 2 { bindings[String(parts[0])] = String(parts[1]) }
        }
        // Simple variable resolution: replace {{key}} with value
        for (key, value) in bindings {
            content = content.replacingOccurrences(of: "{{\({key})}}", with: value)
        }
    }
    print(content)
}
}

```

Step 4: Create PaulStore (SwiftData reader)

```

// PaulCLI/PaultStore.swift
import Foundation
import SwiftData

/// Read-only access to the Pault SwiftData store from CLI context.
final class PaulStore {
    let container: ModelContainer
    let context: ModelContext

    init() throws {
        let appSupport = FileManager.default.urls(for: .applicationSupportDirectory, in:
            .userDomainMask).first!
        let storeURL = appSupport.appendingPathComponent("Pault/default.store")

        let schema = Schema([Prompt.self, Tag.self, TemplateVariable.self,
            Attachment.self, PromptRun.self, CopyEvent.self,
            PromptVersion.self, SmartCollection.self])
        let config = ModelConfiguration(schema: schema, url: storeURL, isStoredInMemoryOnly:
            false)
        container = try ModelContainer(for: schema, configurations: [config])
        context = ModelContext(container)
    }

    var prompts: [Prompt] {
        (try? context.fetch(FetchDescriptor<Prompt>())) ?? []
    }
}

```

```

}

func findPrompt(title: String) -> Prompt? {
    prompts.first(where: { $0.title.localizedCaseInsensitiveContains(title) })
}
}

```

Step 5: Create RunCommand (Pro-gated by Keychain check)

```

// PaultCLI/Commands/RunCommand.swift
import Foundation
import ArgumentParser
import Security

struct RunCommand: ParsableCommand {
    static let configuration = CommandConfiguration(
        commandName: "run",
        abstract: "Run a prompt against the configured LLM (Pro)"
    )

    @Argument(help: "Prompt title")
    var title: String

    @Option(name: .customLong("var"), parsing: .upToNextOption, help: "Variable bindings: key=value")
    var variables: [String] = []

    func run() throws {
        // Check API key is configured (simple Keychain read)
        guard let apiKey = loadKeychainValue(key: "com.pault.api-key.claude") ??
            loadKeychainValue(key: "com.pault.api-key.openai") else {
            throw ValidationError("No API key configured. Open Pault → Preferences → AI to add one.")
        }

        let store = try PaultStore()
        guard let prompt = store.findPrompt(title: title) else {
            throw ValidationError("No prompt found matching '\(title)'")
        }

        var content = prompt.content
        var bindings: [String: String] = [:]
        for pair in variables {
            let parts = pair.split(separator: "=", maxSplits: 1)
            if parts.count == 2 { bindings[String(parts[0])] = String(parts[1]) }
        }
        for (key, value) in bindings {
            content = content.replacingOccurrences(of: "{$\\(key)}", with: value)
        }

        // Run synchronously via URLSession (CLI context)
        let result = runPromptSync(content: content, apiKey: apiKey)
        print(result)
    }

    private func loadKeychainValue(key: String) -> String? {
        let query: [CFString: Any] = [
            kSecClass: kSecClassGenericPassword,

```

```

kSecAttrService: "com.pault.app",
kSecAttrAccount: key,
kSecReturnData: true,
kSecMatchLimit: kSecMatchLimitOne
]
var result: AnyObject?
let status = SecItemCopyMatching(query as CFDictionary, &result)
guard status == errSecSuccess, let data = result as? Data else { return nil }
return String(data: data, encoding: .utf8)
}

private func runPromptSync(content: String, apiKey: String) -> String {
var output = ""
let semaphore = DispatchSemaphore(value: 0)

Task {
var request = URLRequest(url: URL(string: "https://api.anthropic.com/v1/messages")!)
request.httpMethod = "POST"
request.setValue(apiKey, forHTTPHeaderField: "x-api-key")
request.setValue("2023-06-01", forHTTPHeaderField: "anthropic-version")
request.setValue("application/json", forHTTPHeaderField: "Content-Type")
let body: [String: Any] = [
"model": "claude-opus-4-6",
"max_tokens": 4096,
"messages": [{"role": "user", "content": content}]
]
request.httpBody = try? JSONSerialization.data(withJSONObject: body)

if let (data, _) = try? await URLSession.shared.data(for: request),
let json = try? JSONSerialization.jsonObject(with: data) as? [String: Any],
let content = (json["content"] as? [[String: Any]])?.first?["text"] as? String {
output = content
} else {
output = "Error: Failed to get response from API."
}
semaphore.signal()
}

semaphore.wait()
return output
}
}

```

Step 6: Add Xcode target (manual)

In Xcode:

1. File → New → Target → Command Line Tool
2. Name: PaultCLI, Language: Swift
3. Add ArgumentParser package dependency (File → Add Package → <https://github.com/apple/swift-argument-parser>)
4. Add PaultCLI/main.swift and PaultCLI/Commands/*.swift to the target
5. Add Pault framework as dependency so CLI can import Prompt, Tag, etc. — OR copy only the needed models into a shared framework target

Step 7: Build CLI target

```
xcodebuild build -scheme PaultCLI -destination 'platform=macOS' 2>&1 | grep -E
"error:|Build"
```

Step 8: Test CLI commands manually

```
# Assuming the built binary is in DerivedData
BINARY=$(find ~/Library/Developer/Xcode/DerivedData -name "pault" -type f | head -1)

$BINARY list
$BINARY list --favorites --json
$BINARY get "My prompt"
$BINARY run "My prompt" --var topic="Swift concurrency"
```

Step 9: Commit

```
git add PaultCLI/
git commit -m "feat: add PaultCLI executable with list, get, copy, run commands"
```

Verification Checklist

Analytics

- [] Copy a prompt 5 times → Inspector Stats tab shows Copies: 5
- [] Run a prompt → Inspector Stats tab shows Runs: 1
- [] Analytics sheet (chart.bar toolbar button) → Top Prompts table sorted by usage
- [] Sparkline chart in Stats tab reflects 30-day copy history
- [] Free user → Stats tab shows lock badge

Versioning

- [] Edit and save a prompt → Inspector "History" tab → version row appears with timestamp
- [] Select a version → "View Diff" → side-by-side diff shows old vs current content
- [] Select a version → "Restore" → confirm dialog → prompt content reverts
- [] Save 55 versions → verify only 50 remain (pruning)
- [] Free user → History tab shows Pro gate

Smart Collections

- [] Sidebar → New Collection → save a filter → collection appears → filters correctly
- [] "Generate with AI" → AI clusters prompts → multiple collections created
- [] SmartCollection filter .savedFilter respects tags + recency + favorites rules
- [] Free user → Collections section shows lock

CLI

- [] `pault list` → prints all non-archived prompt titles
- [] `pault list --tag writing --json` → JSON array filtered by tag
- [] `pault get "Prompt title"` → prints prompt content
- [] `pault run "Prompt title" --var key=value` → LLM response printed to stdout
- [] Pipe test: `echo "text" | pault run "summarize" --var text=-` (if stdin support added)